

SQL Syntax Cheat Sheet — Complete Reference

DENSE REFERENCE: syntax boxes + one-liner examples. Every major SQL construct.

DDL — Data Definition Language

CREATE TABLE

```
CREATE TABLE table_name (  
    col_name    data_type    [CONSTRAINT constraint_name] constraint_def,  
    ...  
    [table_constraints]  
);
```

```
CREATE TABLE orders (  
    order_id    BIGSERIAL PRIMARY KEY,  
    customer_id BIGINT      NOT NULL REFERENCES customers(customer_id),  
    total       NUMERIC(12,2) CHECK (total >= 0),  
    status      VARCHAR(20)  DEFAULT 'pending',  
    created_at  TIMESTAMPTZ  DEFAULT NOW()  
);
```

CREATE TABLE ... AS (CTAS)

```
CREATE TABLE archive_orders AS SELECT * FROM orders WHERE created_at < '2023-01-01';
```

ALTER TABLE

```
ALTER TABLE t ADD COLUMN col data_type;  
ALTER TABLE t DROP COLUMN col;  
ALTER TABLE t ALTER COLUMN col TYPE new_type [USING expr];  
ALTER TABLE t ALTER COLUMN col SET DEFAULT val;  
ALTER TABLE t ALTER COLUMN col SET NOT NULL;  
ALTER TABLE t ALTER COLUMN col DROP NOT NULL;  
ALTER TABLE t RENAME COLUMN old TO new;  
ALTER TABLE t ADD CONSTRAINT name constraint_def;  
ALTER TABLE t DROP CONSTRAINT name;  
ALTER TABLE t RENAME TO new_name;
```

```
ALTER TABLE users ADD COLUMN last_login TIMESTAMPTZ;  
ALTER TABLE products ALTER COLUMN price TYPE NUMERIC(10,2);
```

DROP

```
DROP TABLE [IF EXISTS] t [CASCADE | RESTRICT];
DROP INDEX [IF EXISTS] idx;
DROP VIEW [IF EXISTS] v [CASCADE];
```

TRUNCATE

```
TRUNCATE TABLE t [RESTART IDENTITY] [CASCADE];
```

Constraints

Constraint	Syntax	Notes
PRIMARY KEY	col TYPE PRIMARY KEY	Unique + NOT NULL
FOREIGN KEY	`REFERENCES parent(col) [ON DELETE CASCADE	SET NULL
UNIQUE	col TYPE UNIQUE OR UNIQUE(col1, col2)	Allows NULLs (NULLs not equal)
NOT NULL	col TYPE NOT NULL	No NULLs
CHECK	CHECK (condition)	Validate at write time
DEFAULT	DEFAULT expr	Value when not supplied
EXCLUDE	EXCLUDE USING gist (col WITH op)	PostgreSQL: no overlapping ranges

```
-- Composite unique
ALTER TABLE order_items ADD CONSTRAINT uq_order_item UNIQUE (order_id, product_id);

-- Check constraint
ALTER TABLE products ADD CONSTRAINT chk_price CHECK (price > 0);

-- FK with cascade delete
ALTER TABLE order_items ADD FOREIGN KEY (order_id) REFERENCES orders(order_id) ON
DELETE CASCADE;
```

Data Types Quick Reference

Category	Types
Integer	SMALLINT (2B), INT/INTEGER (4B), BIGINT (8B), BIGSERIAL (auto-inc)
Decimal	NUMERIC(p,s), DECIMAL(p,s) — exact; FLOAT/REAL — approximate
Text	CHAR(n), VARCHAR(n), TEXT (unlimited)
Date/Time	DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL
Boolean	BOOLEAN (true/false/null)

UUID	UUID
JSON	JSON (text-stored), JSONB (binary, indexable)
Array	type[] e.g. INT[], TEXT[]
Range	INT4RANGE, TSRANGE, TSTZRANGE, DATERANGE
Network	INET, CIDR, MACADDR
Geometric	POINT, LINE, CIRCLE, POLYGON
Binary	BYTEA
Full-text	TSVECTOR, TSQUERY

GOTCHA: Use *NUMERIC* for money, never *FLOAT* (precision loss). Use *BIGINT* for IDs, not *INT* (2B limit). Use *TIMESTAMPZ* (stores UTC), not *TIMESTAMP* (no timezone info).

DML — Data Manipulation Language

INSERT

```
INSERT INTO t (col1, col2) VALUES (v1, v2), (v3, v4);
INSERT INTO t (col1, col2) SELECT ... FROM other;
INSERT INTO t DEFAULT VALUES;

-- UPSERT (PostgreSQL)
INSERT INTO t (col1, col2) VALUES (v1, v2)
ON CONFLICT (col1) DO UPDATE SET col2 = EXCLUDED.col2;

INSERT INTO t (col1, col2) VALUES (v1, v2)
ON CONFLICT DO NOTHING;
```

UPDATE

```
UPDATE t SET col1 = v1, col2 = v2 WHERE condition;
UPDATE t SET col1 = v1 FROM other_table WHERE t.id = other_table.id;

-- Returning modified rows
UPDATE orders SET status = 'shipped' WHERE order_id = 42 RETURNING *;
```

DELETE

```
DELETE FROM t WHERE condition;
DELETE FROM t USING other WHERE t.id = other.id;
DELETE FROM t WHERE id IN (SELECT id FROM other);
```

RETURNING clause (PostgreSQL)

```
INSERT INTO users (name, email) VALUES ('Alice', 'a@b.com') RETURNING user_id,
created_at;
UPDATE inventory SET qty = qty - 1 WHERE product_id = 5 RETURNING qty AS new_qty;
DELETE FROM sessions WHERE expires_at < NOW() RETURNING session_id;
```

DQL — Data Query Language

SELECT Anatomy

```
SELECT  [DISTINCT] columns / expressions / *
FROM    table [alias]
        [JOIN ...]
WHERE   conditions                -- filters rows BEFORE grouping
GROUP BY grouping_cols
HAVING  conditions_on_groups      -- filters groups AFTER grouping
ORDER BY sort_cols [ASC|DESC] [NULLS FIRST|LAST]
LIMIT  n
OFFSET m;
```

GOTCHA: **Logical execution order** (not written order): FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET

JOINS

Types Reference

Join Type	Returns
INNER JOIN / JOIN	Rows with match in BOTH tables
LEFT JOIN / LEFT OUTER JOIN	All from left + matching right (NULL if no match)
RIGHT JOIN	All from right + matching left
FULL OUTER JOIN	All from both, NULLs where no match
CROSS JOIN	Cartesian product (every combo)
NATURAL JOIN	Implicit join on same-named columns (avoid in prod)
LATERAL JOIN	Right side can reference left side columns
SELF JOIN	Join table to itself

```
-- INNER JOIN
SELECT o.order_id, c.name FROM orders o JOIN customers c ON o.customer_id =
c.customer_id;

-- LEFT JOIN (customers with no orders)
SELECT c.name, COUNT(o.order_id) FROM customers c LEFT JOIN orders o ON
```

```

c.customer_id = o.customer_id GROUP BY c.name;

-- SELF JOIN (employees + manager name)
SELECT e.name, m.name AS manager FROM employees e LEFT JOIN employees m ON
e.manager_id = m.employee_id;

-- LATERAL (top 3 orders per customer)
SELECT c.name, latest.* FROM customers c
CROSS JOIN LATERAL (SELECT * FROM orders WHERE customer_id = c.customer_id ORDER BY
created_at DESC LIMIT 3) latest;

```

GOTCHA: `FULL OUTER JOIN` with `WHERE a.id IS NULL OR b.id IS NULL` finds unmatched rows in either table.

Aggregation Functions

Function	Example	Notes
COUNT(*)	COUNT(*)	All rows
COUNT(col)	COUNT(email)	Non-NULL values
COUNT(DISTINCT col)		Slow on large sets — consider HLL
SUM(col)	SUM(amount)	NULLs ignored
AVG(col)	AVG(price)	NULLs ignored
MIN(col)		
MAX(col)		
STRING_AGG(col, delim)	STRING_AGG(name, ', ')	Concatenate strings
ARRAY_AGG(col)		Collect into array
BOOL_AND(col)		True if all true
BOOL_OR(col)		True if any true
PERCENTILE_CONT(f) WITHIN GROUP (ORDER BY col)	PERCENTILE_CONT(0.5)...	Median / percentiles
PERCENTILE_DISC(f) WITHIN GROUP (ORDER BY col)		Discrete (actual value)
MODE() WITHIN GROUP (ORDER BY col)		Most frequent value
CORR(col1, col2)		Pearson correlation (-1 to 1)

```

-- FILTER clause (conditional aggregation)
SELECT
    COUNT(*) FILTER (WHERE status = 'completed') AS completed,

```

```

    SUM(amount) FILTER (WHERE status = 'completed') AS revenue
FROM orders;

-- GROUP BY ROLLUP (subtotals)
SELECT region, country, SUM(sales) FROM sales_data GROUP BY ROLLUP (region,
country);

-- GROUP BY CUBE (all combinations)
SELECT region, product, SUM(sales) FROM sales_data GROUP BY CUBE (region, product);

-- GROUP BY GROUPING SETS (specific combinations)
SELECT region, product, SUM(sales) FROM sales_data GROUP BY GROUPING SETS ((region),
(product), ());

```

Window Functions

Syntax

```

function_name(args) OVER (
    [PARTITION BY col1, col2]
    [ORDER BY col3 [ASC|DESC]]
    [ROWS|RANGE|GROUPS BETWEEN start AND end]
)

```

Frame Options

Frame	Meaning
UNBOUNDED PRECEDING	Start of partition
n PRECEDING	n rows/range before current
CURRENT ROW	Current row
n FOLLOWING	n rows/range after current
UNBOUNDED FOLLOWING	End of partition

Ranking Functions

```

ROW_NUMBER() OVER (...)          -- 1,2,3,4 (no ties)
RANK() OVER (...)                 -- 1,1,3,4 (gaps on ties)
DENSE_RANK() OVER (...)          -- 1,1,2,3 (no gaps)
NTILE(n) OVER (...)              -- divide into n buckets
PERCENT_RANK() OVER (...)        -- (rank-1)/(rows-1) [0.0 to 1.0]
CUME_DIST() OVER (...)           -- cumulative distribution [0 to 1]

```

Offset Functions

```
LAG(col, [n], [default]) OVER (...) -- previous n-th row value
LEAD(col, [n], [default]) OVER (...) -- next n-th row value
FIRST_VALUE(col) OVER (...) -- first value in frame
LAST_VALUE(col) OVER (...) -- last value in frame (check frame!)
NTH_VALUE(col, n) OVER (...) -- nth value in frame
```

GOTCHA: `LAST_VALUE` uses default frame `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — which makes it return the current row's value. Specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to get the partition's last value.

Examples

```
-- Running total
SUM(amount) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)

-- Percentage of total
SUM(amount) OVER (PARTITION BY category) / SUM(amount) OVER () * 100

-- 7-day moving average
AVG(value) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

-- Rank within group
RANK() OVER (PARTITION BY department ORDER BY salary DESC)

-- Previous row value
LAG(price) OVER (PARTITION BY product_id ORDER BY price_date)

-- First value in partition
FIRST_VALUE(name) OVER (PARTITION BY dept ORDER BY salary DESC)
```

CTEs — Common Table Expressions

Regular CTE

```
WITH cte_name AS (
    SELECT ...
),
cte2 AS (
    SELECT ... FROM cte_name
)
SELECT * FROM cte2;
```

Recursive CTE

```
WITH RECURSIVE cte AS (
    -- Base case
    SELECT id, parent_id, name, 0 AS depth FROM tree WHERE parent_id IS NULL
```

```

UNION ALL
-- Recursive step
SELECT t.id, t.parent_id, t.name, cte.depth + 1
FROM tree t
JOIN cte ON t.parent_id = cte.id
)
SELECT * FROM cte ORDER BY depth, name;

```

Materialized CTE (PostgreSQL 12+)

```

WITH expensive AS MATERIALIZED (
    SELECT ... expensive computation ...
)
SELECT * FROM expensive WHERE ...;
-- Forces materialization (computed once, reused)
-- Default in PG12+: CTEs are inlined (NOT materialized)

```

GOTCHA: Pre-PostgreSQL 12, ALL CTEs were optimization fences (always materialized). PG12+ inlines CTEs by default. Use `MATERIALIZED` keyword to force old behavior.

Subqueries

Types

```

-- Scalar subquery (returns one value)
SELECT name, salary, (SELECT AVG(salary) FROM employees) AS avg_sal FROM employees;

-- Row subquery
SELECT * FROM employees WHERE (department_id, salary) = (SELECT dept_id, max_sal
FROM dept_max);

-- Table subquery (in FROM)
SELECT * FROM (SELECT *, ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn FROM
employees) ranked WHERE rn <= 5;

-- Correlated subquery (references outer query)
SELECT e.name FROM employees e WHERE e.salary > (SELECT AVG(salary) FROM employees
e2 WHERE e2.department_id = e.department_id);

-- EXISTS
SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id =
c.customer_id);

-- NOT EXISTS (safer than NOT IN with NULLs)
SELECT * FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE
o.customer_id = c.customer_id);

-- IN
SELECT * FROM products WHERE category_id IN (SELECT category_id FROM categories

```



```
WHERE name = 'Electronics');
```

```
-- ANY / ALL
```

```
SELECT * FROM employees WHERE salary > ANY (SELECT salary FROM managers);
```

```
SELECT * FROM employees WHERE salary > ALL (SELECT salary FROM managers);
```

GOTCHA: `NOT IN` returns NO ROWS if the subquery contains any NULL values. Always prefer `NOT EXISTS`.

Set Operations

```
query1 UNION query2          -- combine, removes duplicates
query1 UNION ALL query2      -- combine, keeps duplicates (faster)
query1 INTERSECT query2      -- rows in both
query1 INTERSECT ALL query2
query1 EXCEPT query2       -- rows in left but not right
query1 EXCEPT ALL query2
```

Rules:

- Same number of columns
- Compatible data types
- ORDER BY applies to final result only

```
-- Customers in Q1 but not Q2
SELECT customer_id FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31'
EXCEPT
SELECT customer_id FROM orders WHERE order_date BETWEEN '2024-04-01' AND '2024-06-30';
```

String Functions

```
LENGTH(str)                -- 5
UPPER(str) / LOWER(str)     -- 'HELLO' / 'hello'
TRIM(str) / LTRIM / RTRIM   -- remove whitespace
SUBSTR(str, start, len)     -- 'hello' → SUBSTR('hello world',1,5)
LEFT(str, n) / RIGHT(str, n) -- first/last n chars
REPLACE(str, from, to)      -- replace substring
REGEXP_REPLACE(str, pattern, rep) -- regex replace
LIKE / ILIKE (case-insensitive) -- pattern matching with % and _
CONCAT(a, b, ...) or a || b -- concatenation
SPLIT_PART(str, delim, n)   -- 'a,b,c' → SPLIT_PART('a,b,c',',',2) = 'b'
LPAD(str, len, pad)        -- pad left
RPAD(str, len, pad)        -- pad right
FORMAT('%s has %s items', name, n) -- printf-style formatting
INITCAP(str)                -- 'hello world' → 'Hello World'
REPEAT(str, n)              -- repeat string n times
```

Date/Time Functions

```
NOW()                -- current timestamp with TZ
CURRENT_DATE         -- current date
CURRENT_TIME         -- current time
EXTRACT(part FROM date)  -- EXTRACT(YEAR FROM NOW())
DATE_PART('year', date)  -- same as EXTRACT
DATE_TRUNC('month', date) -- truncate to month start
AGE(ts1, ts2)          -- interval between two timestamps
INTERVAL 'n unit'      -- INTERVAL '3 days', '2 hours'
date + INTERVAL        -- date arithmetic
TO_DATE(str, format)    -- TO_DATE('2024-01-15', 'YYYY-MM-DD')
TO_TIMESTAMP(str, format) -- TO_TIMESTAMP('01/15/2024', 'MM/DD/YYYY')
TO_CHAR(date, format)   -- TO_CHAR(NOW(), 'YYYY-MM-DD HH24:MI:SS')
AT TIME ZONE 'UTC'      -- convert to timezone
GENERATE_SERIES(start, end, step) -- generate date range
```

```
-- Date spine (all days in a range)
SELECT generate_series('2024-01-01'::date, '2024-12-31'::date, '1
day'::interval)::date AS day;

-- Days between dates
SELECT '2024-06-01'::date - '2024-01-01'::date AS days; -- 152

-- First/last day of month
DATE_TRUNC('month', NOW()) -- first day
DATE_TRUNC('month', NOW()) + INTERVAL '1 month' - INTERVAL '1 day' -- last day
```

NULL Handling

```
IS NULL / IS NOT NULL  -- test for NULL
COALESCE(a, b, c)       -- return first non-NULL
NULLIF(a, b)            -- return NULL if a = b, else return a
GREATEST(a, b) / LEAST(a, b) -- max/min, ignoring NULLs
NVL(a, b)               -- NOT PostgreSQL; use COALESCE
```

GOTCHA: Any arithmetic with NULL returns NULL. `NULL = NULL` is NULL, not TRUE. Use `IS NULL` to test. `NOT IN` with NULLs in subquery returns no rows.

```
COALESCE(revenue, 0)      -- replace NULL with 0
NULLIF(denominator, 0)    -- avoid division by zero (returns NULL instead)
GREATEST(last_login, created_at) -- returns non-null when one is null
```

CASE Expression

```
-- Simple CASE
CASE col
  WHEN val1 THEN result1
  WHEN val2 THEN result2
  ELSE default_result
END

-- Searched CASE
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

```
-- Conditional aggregation
SELECT
  SUM(CASE WHEN status = 'completed' THEN amount ELSE 0 END) AS completed_revenue,
  COUNT(CASE WHEN status = 'cancelled' THEN 1 END) AS cancellations
FROM orders;

-- Bucket/bin values
SELECT
  CASE
    WHEN age < 18 THEN 'minor'
    WHEN age BETWEEN 18 AND 64 THEN 'adult'
    ELSE 'senior'
  END AS age_group
FROM users;
```

DISTINCT and DISTINCT ON

```
SELECT DISTINCT col FROM table;           -- unique values of col
SELECT DISTINCT col1, col2 FROM table;    -- unique combinations

-- DISTINCT ON (PostgreSQL only): keep first row per group
SELECT DISTINCT ON (customer_id)
  customer_id, order_id, order_date
FROM orders
ORDER BY customer_id, order_date DESC; -- ORDER BY must include DISTINCT ON cols
-- Returns latest order per customer
```

LIMIT, OFFSET, FETCH

```
LIMIT n                -- return max n rows
OFFSET m               -- skip first m rows
LIMIT n OFFSET m       -- pagination: page p has OFFSET (p-1)*n, LIMIT n
```

```
-- SQL Standard syntax (PostgreSQL supports both)
FETCH FIRST n ROWS ONLY
OFFSET m ROWS FETCH FIRST n ROWS ONLY
```

GOTCHA: OFFSET becomes extremely slow on large tables (DB must scan and discard). Use keyset pagination for large datasets:

```
-- Keyset pagination (efficient)
SELECT * FROM orders WHERE order_id > :last_seen_id ORDER BY order_id LIMIT 20;
```

Full-Text Search (PostgreSQL)

```
-- Convert text to tsvector
TO_TSVECTOR('english', 'The quick brown fox')
-- Weights: setweight(ts, 'A'|'B'|'C'|'D')

-- Create query
PLAIN_TO_TSQUERY('english', 'quick fox') -- AND logic
TO_TSQUERY('english', 'quick & fox')      -- AND
TO_TSQUERY('english', 'quick | fox')       -- OR
PHRASE_TO_TSQUERY('english', 'quick fox')  -- phrase

-- Match
tsvector @@ tsquery

-- Rank results
TS_RANK(tsvector, tsquery)                -- TF-based rank
TS_RANK_CD(tsvector, tsquery)              -- cover density rank

-- Highlight matches
TS_HEADLINE('english', text, query)
```

```
-- Index for FTS
CREATE INDEX idx_articles_fts ON articles USING GIN (TO_TSVECTOR('english', title ||
' ' || body));
-- Or: GENERATED ALWAYS AS column + GIN index
```

JSONB Operations (PostgreSQL)

```
-- Access
data -> 'key'          -- get JSON value (JSON type)
data ->> 'key'         -- get as text
data #> '{a,b}'        -- nested path (JSON)
data #>> '{a,b}'       -- nested path (text)
```

```

-- Test
data ? 'key' -- key exists
data ?| ARRAY['a','b'] -- any key exists
data ?& ARRAY['a','b'] -- all keys exist
data @> '{"key":"val"}' -- contains

-- Modify
data || '{"new_key":"val"}'::jsonb -- merge/add key
data - 'key' -- remove key
JSONB_SET(data, '{key}', '"value"') -- set nested path

-- Expand
JSONB_EACH(data) -- expand to (key, value) rows
JSONB_ARRAY_ELEMENTS(array_col) -- expand JSON array
JSONB_OBJECT_KEYS(data) -- get all keys
JSONB_PATH_QUERY(data, '$.items[*]') -- jsonpath query

-- Index
CREATE INDEX idx_data ON table USING GIN (jsonb_col);
CREATE INDEX idx_specific ON table ((jsonb_col->>'status')); -- specific key

```

Arrays (PostgreSQL)

```

-- Literal
ARRAY[1, 2, 3]
ARRAY['a', 'b', 'c']
'{1,2,3}'::int[]

-- Access (1-indexed!)
arr[1] -- first element
arr[2:4] -- slice

-- Functions
ARRAY_LENGTH(arr, 1) -- length of dimension 1
ARRAY_UPPER(arr, 1) -- upper bound
CARDINALITY(arr) -- total elements
UNNEST(arr) -- expand to rows
ARRAY_APPEND(arr, v) -- append element
ARRAY_PREPEND(v, arr) -- prepend element
ARRAY_CAT(arr1, arr2) -- concatenate
arr1 || arr2 -- concatenate (operator)
v = ANY(arr) -- element in array
v <> ALL(arr) -- element not in array
arr1 @> arr2 -- arr1 contains arr2
ARRAY_AGG(col) -- aggregate column to array
ARRAY_REMOVE(arr, v) -- remove all occurrences of v

```

WINDOW vs GROUP BY Comparison

	GROUP BY	Window Function
Reduces rows	Yes	No (keeps all rows)
Can reference non-agg cols	No (only grouped)	Yes
Can access other rows	No	Yes (LAG, LEAD, frame)
Multiple aggregations per query	Yes	Yes
Combined with GROUP BY	Yes (HAVING)	Yes (window operates on grouped result)